Chemical Engineering Science 216 (2020) 115467

Contents lists available at ScienceDirect

Chemical Engineering Science

journal homepage: www.elsevier.com/locate/ces

BubCNN: Bubble detection using Faster RCNN and shape regression network

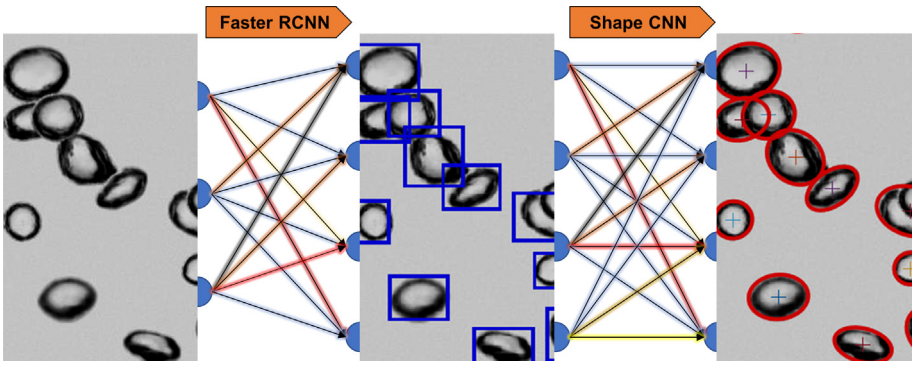
Tim Haas *, Christian Schubert, Moritz Eickhoff, Herbert Pfeifer

Department of Industrial Furnaces and Heat Engineering, RWTH Aachen University, Kopernikusstraße 10, D–52074 Aachen, Germany

HIGHLIGHTS

- Development of a bubble detection method based on machine learning named BubCNN.
- Includes a Faster RCNN detector module and a regression CNN for shape approximation.
- Optimal Hyperparameters and architecture for both modules were analyzed.
- BubCNN yield accurate results and show great potential for generalization.
- Detector and a Transfer Learning Module are made publicly available on GitHub.

GRAPHICAL ABSTRACT



ARTICLE INFO

Article history:
Received 23 October 2019
Received in revised form 12 December 2019
Accepted 31 December 2019
Available online 7 January 2020

Keywords:
Bubble detection
Machine learning
Convolutional neural networks
Image processing

ABSTRACT

Detailed knowledge about gas-liquid multiphase flows is important to optimize industrial systems. Imaging with image processing is the most commonly used measurement technique. However, the workflow and parameters strongly depend on the experimental conditions and no generally applicable process has been developed yet. Here, a workflow based on convolutional neural networks (CNN) is proposed that can be used with a wider range of experimental conditions. The method, named BubCNN, employs a Faster region-based CNN (RCNN) detector to locate bubbles and a shape regression CNN to predict bubble shape parameters. Hyperparameters and network architectures for both modules were systematically analyzed. BubCNN achieved accurate results for different experimental conditions. A pretrained program was made publicly available on GitHub. Since the whole variety of bubble images was not yet captured in the training data set, an additional semi-automatic transfer learning module is provided that allows to customize BubCNN for different images.

1. Introduction

Gas stirring is an integral part in numerous process units. Purging gas is used in case mechanic stirring is prohibited by high temperature or reactive flows or the usage of gas is cheaper and more reliable. In a steel casting ladle, inert argon gas is introduced to promote a homogenization of alloying elements and temperature. Beyond that, reactive gas is sometimes used. For example, in aluminum refinement, chlorine gas is introduced to remove hydrogen from the melt.

For process optimization, detailed knowledge about the bubble characteristics and dynamics is crucial. The bubble size distribution is important for numerical flow models as well as thermodynamic reaction models. The interaction between the bubbles and the liquid determines the quantity of transferred mixing energy.

* Corresponding author.
E-mail address: haas@iob.rwth-aachen.de (T. Haas).

https://doi.org/10.1016/j.ces.2019.115467
0009-2509/

Furthermore, the bubble rising velocity can be used to validate numerical models (Haas and Eickhoff, 2019) and determines the residence time and mass transfer rate for reaction models.

Gaining this information however can be quite challenging. This is mainly due to the large variety of factors that can restrict or influence the measurements. Owing to that, hitherto methods were only applicable for given experimental conditions but could not be applied in altered situations.

Different methods for evaluating bubble characteristics in a fluid medium can be found in the literature. They can be separated into intrusive and non-intrusive methods. Intrusive methods, such as conductivity probes or fiber optic probes, interact to some extent with the flow and are usually restricted to single or few bubbles. Thus, the results may be biased by the applied measurement technique. On the other hand, intrusive methods can be applied in a broader range of setups and with different liquids. Non-intrusive methods do not affect the flow but have more constrains regarding their applicability. Amongst the non-intrusive methods, direct imaging is most frequently used because multiple objects can be observed simultaneously and the equipment is comparably cheap and flexible.

An extraction of the bubble location and shape from the images is usually achieved by using multi-staged approaches of pixel intensity level analysis by means of traditional computer vision techniques (Fu and Liu, 2016; Bröder and Sommerfeld, 2007; Ferreira et al., 2012; Karn et al., 2015; Lau et al., 2013). A particular problem is that the bubbles may overlap on the 2D projection and are visible as a cluster of bubbles.

Image processing usually starts with a boundary extraction by either applying a global or local threshold, background subtraction or edge detection by analyzing the local intensity gradients. In the second stage, the boundaries of objects, that may be individual bubbles or bubble clusters are split into curve segments of bubbles. That can be achieved by seed-point extraction or breakpoint-based methods. Seed-points are defined as the centers of overlapping objects and are used to determine the number of bubbles in a cluster. Different techniques were used, namely bounded erosion with fast radial symmetry transform (Zafari et al., 2015), ultimate erosion (Hukkanen et al., 2010) and elliptical distance transform (Talbot and Appleton, 2002). Breakpoint methods segment the contour of a cluster into the contour of individual objects by detecting the points where the outlines meet. This can be done by analyzing the convexity of the outline (Zafari et al., 2016) or by a combination of boundary curvature and intensity gradient analysis (Fu and Liu, 2016). The curve segments need to be regrouped so that different segments of the same bubble are clustered. Again, different methods are available such as k-mean clustering (Kaur and Mittal, 2017) or watershed (Fu and Liu, 2016). Often the bubble shape is finally approximated by using the grouped curve segments for an ellipse fitting.

A major problem of the traditional imaging techniques is that the parameters of the different stages and even the workflow itself strongly depend on the experimental settings at which the images were taken. Experimental variables such as the distance between the bubbles and the camera or the background light intensity significantly influence the applicability of these techniques. However, these variables are often constrained by experimental conditions. The potential to generalize the workflows is low, so a lot of manual adjustment is necessary that requires expert knowledge in image processing. A universally applicable approach was not found hitherto.

A relatively novel alternative approach to traditional computer vision techniques are convolutional neural networks (CNN). It is based on analyzing pixel intensity levels too, but in contrast to the conventional methods, convolutional neural networks do not rely on manually extracted features but learn to extract these features based on a labeled training data set. CNNs have pushed the ability of computer vision to astonishing levels within the last years. For classification tasks CNNs have outperformed traditional methods easily and the error rates of the latest developments are even below those of humans (Russakovsky et al., 2015).

So far, CNNs have been rarely used for the analysis of bubble swarms. Poletaev et al. (Poletaev et al., 2016) used a sliding window approach. Thereby a small image patches is extracted from the upper left corner of a bubble swarm image and is processed by a classification CNN to figure out whether that patch contains a bubble or not. If it does, another CNN is used to output ellipse parameters that describe the shape of the bubble. Afterwards the window is shifted by a few pixels and the process is repeated. The CNNs were trained by a training data set made of synthetic bubble images. It was found that CNNs achieve much better results compared to regular perceptrons and that the Adam training algorithm yields better results in terms of accuracy and convergence compared to stochastic gradient descent (SGD), SGD-Nesterov, AdaDelta and AdaGrad. The missing rate of the system was about 5–20 percent, depending on the number of pixels the image patch was shifted between the passes.

Ilonen et al. (Ilonen et al., 2018) used a multiscale sliding window approach for bubble detection. Here a single classification network is trained and then applied to the same image that is resized multiple times. By that, bubbles with different sizes can be detected. However, the network is only applicable for spherical bubbles and the computational effort is significant.

In this work, a novel CNN based method is introduced. The approach identifies bubbles and their shapes by a two-stage procedure. Because parts of the work were inspired by Fu's and Liu's BubGAN (Fu and Liu, 2019), it is named BubCNN. In the first step a Faster RCNN detector is used to locate the bubbles. Subsequently the identified regions are extracted and processed by a shape regression CNN to approximate the bubble shape by an ellipse. By that, the detector combines the efficiency of modern Faster region-based convolutional neural network (RCNN) detectors and the accuracy of regression CNNs. The main advantage of that technique is that it is more general than traditional imaging techniques and can be applied to a broader range of experimental conditions. In addition, it can be used without expert knowledge about image processing.

2. Convolutional neural networks

In the following section, the function of the most important network layers is explained. Though some additional, less frequently used layers exist, a fully functional convolutional neural network can be constructed by stacking these layers.

2.1. Image input layer

For a computer, images are represented by two- or three-dimensional matrices of integer numbers. This numbers range from 0 to 255 representing the light intensity or the intensity of green, blue and red in a pixel. All CNNs start with an image input layer in which images are loaded into the network. Usually, the image input layer also conducts an image zero-center normalization. Thereby, the mean pixel values for the whole training set are subtracted to make the network less sensitive to different illuminations. The height and width of the input images are defined in that layer as well as the number of channels. For a grayscale image, there is only one channel, for color images, there are three. The subsequent convolutional, activation and pooling layers perform mathematical operations which are independent of the image size. However, the final fully connected layers, require a fixed number of

neurons. Thus, Faster RCNNs can process images of arbitrary size while classification or regression CNNs require images with a fixed height and width.

2.2. Convolutional layers

Convolutional layers compute a two- or three-dimensional convolutional operation. For a single channel input, the operation is given by:

$$z = \begin{bmatrix} w_{ij} & w_{ij} & w_{ij} \\ w_{ij} & w_{ij} & w_{ij} \\ w_{ij} & w_{ij} & w_{ij} \end{bmatrix}^{\circ} R$$

where $\circ$ denotes a convolution operation which is the sum of the elementwise multiplication, z is a single real number in the output feature map, w_{ij} are the weights of the filter and R is the receptive field. The receptive field is a section of the convolutional layers input which slide through all its rows and columns. Filter and receptive field both have the same depth. For example, if the input image is a three-channeled RGB image, the filter also has a depth of three. If the first convolutional layer consists of sixteen individual filters, the depth of filters in the subsequent convolutional layer is also sixteen. All filter weights are initialized by small random numbers and are updated during training by backpropagation. The filters in the first convolutional layers are trained to detect low-level features such as edges, points or areas of specific pixel values. Subsequent convolutional layers combine these features to more abstract features. One filter is trained to detect one specific feature. Thus, a convolutional layer usually contains multiple different filters.

2.3. Activation layer

Though simple to compute, the usage of activation layers is the major advantage of neural networks. Their application achieves a non-linearity of the network that can represent highly complex correlations. An activation layer follows each convolution layer. Thus, a convolutional and an activation layer are often described as one single layer. The first CNNs used sigmoid functions to introduce non-linearity. However, modern network architectures usually use rectifier linear units (ReLu), because it reduces the computational cost during training without major impact on accuracy. The ReLu layer is thresholding the feature map by:

$$a = \max(0, z)$$

where a is an output of the ReLu layer and z is an output of the preceding convolutional layer.

2.4. Pooling layers

Pooling layers are used to reduce the dimension of the network. Nowadays, the most commonly used pooling is maximum pooling. Thereby, the preceding feature map is divided into number of sections with defined pooling size, for instance 2×2 matrixes. The maximum pooling keeps only the activation of the neuron with the strongest activation. By that, the storage and computational costs of the network can be reduced and the tendency for overfitting is lowered. In addition, the network gets more shift invariant. The intuition behind pooling layers is that the existence of a feature and its relative position to other features are more important than its size or the absolute distance.

2.5. Fully connected layer

After a sequence of different convolutional, activation and pooling layers, the dimension of the output has usually been reduced significantly while its depth has increased. Each output channel contains information about some specific high-level features that are important for the subsequent classification or regression. The output is reshaped into a vector that is the input for a classical artificial neural network (ANN). All entries of the input vector are connected to all neurons of a hidden layer. The values in each neuron are computed by multiplying the value in the input vector by a weight that has been iteratively adjusted during training. Thereafter, the neurons value is fed into an activation function, again introducing non-linearity.

2.6. Output layer

For classification problems, the output layer has the same size as the number of classes the CNN is able to identify. It computes probabilities for each class, based on the activations of the last hidden layer. In regression, the last layer outputs real numbers.

2.7. Training

The weights of the network are adjusted iteratively by forward-backward propagation. For accurate learning, a large training data set is mandatory. The data set consists of input images and manually created outputs, for example class labels. During learning, a labeled training image is first processed by the network, so that values for all outputs are computed. This is compared to the target values. Then, a loss function is used as performance measure to check how well the network has made the prediction. The deviation between prediction and target is tracked backwards through the network. By that gradient-based optimization, the overall loss can be reduced. This procedure is repeated for all training images multiple times, slowly adjusting the weights. A complete forward and backward processing of all images of the training dataset is called epoch. A complete training usually includes multiple epochs.

2.8. Resources

During the learning process, a considerable number of mathematical operations are executed. Therefore, learning on standard central processing units (CPU) runs very slowly. A significant acceleration can be achieved by using graphics processing units (GPU). For that, the calculation is strongly parallelized. Most object-oriented programming languages that are used for programming neural networks have an automatic translation to CUDA built in. This allows calculations to be performed on NVIDIA GPUs. All computations presented in this work were conducted on the GPU cluster of RWTH Aachen University using the MATLAB framework including the Computer Vision Toolbox, Machine Learning Toolbox and Parallel Computing Toolbox.

3. BubCNN workflow

In contrast to traditional image processing methods, machine learning based methods do not rely on manually extracted features, but the program learns to recognize gas bubbles from training data. Currently available methods do not have the ability to reconstruct the location and shape simultaneously on arbitrary sized images. Thus, the process needs to be split into separate tasks. First the position of bubbles has to be detected. Then the shape of the bubble can be reconstructed. There are different approaches to solve the problem with CNNs in general. As

described above, Poletaev et al. (Poletaev et al., 2016) used a sliding window approach. Independent image patches are first processed by a classification CNN and afterwards by a regression CNN. However, this results in several problems. On the one hand the system is very inefficient, since numerous image patches are processed individually by different CNNs. In addition, it is possible that several bubbles can be visible within a window patch. This problem is particularly pronounced with dense swarms. Thus, a classifier must be able to recognize the number of bubbles. For each number of bubbles, an individually trained regression CNN must be available. A problem here is that the number of arrangements increases strongly with increasing number of bubbles. Given that, regression CNNs almost always have very low accuracies. In preliminary investigations using this method, satisfactory results could only be achieved for low void fractions.

Instead, for BubCNN a two-stage approach consisting of a shape regression CNN and a Faster RCNN detector is proposed. The training and the processing proceed in reversed directions as indicated in Fig. 1. During training, a regression CNN is trained first which is able to extract ellipse parameters from image patches that approximate the bubble shape. In the second training stage, a Faster RCNN is constructed by replacing the final layers of the regression CNN by RCNN specific layers described in detail below. This module is used to detect bubble location on the image. By using the pretrained layers of the regression CNN, significantly higher accuracies can be achieved compared to regular pretrained models such as AlexNet or VGG16. A Faster RCNN provides bounding boxes that do not describe the bubble shape. Therefore, image processing is conducted in reverse direction. First, the Faster RCNN module is used to locate bubbles, then the resulting bounding boxes are extracted, resized and processed by the shape regression CNN. The different process stages are explained in detail below.

3.1. Shape regression CNN module

In many processes the shape of rising gas bubbles can be approximated by an ellipsoid. The validity of this assumption can be determined using the ratio of buoyancy force and surface tension (Eotvos number) and the ratio of inertia and viscous drag force (bubble Reynolds number). The equivalent bubble diameter is considered as characteristic length and the relative velocity between bubble and fluid as characteristic velocity. The assumption of an ellipsoidal shape applies up to a Eotvos number of about 40 while the deviation increases with increasing Eotvos and Reynolds number (Clift et al., 2005).

For small Eotvos and Reynolds numbers the special case of a spherical bubble applies, which can be described however by an ellipsoid with equal semi-axes. In 2D projection, ellipsoids become ellipses which are described mathematically by:

$$\frac{[(x-z_1)\cos(\theta)+(y-z_2)\sin(\theta)]^2}{a^2} + \frac{[(x-z_1)\sin(\theta)+(y-z_2)\cos(\theta)]^2}{b^2} = 1$$

where z_1 and z_2 are the ellipse center, a and b are the ellipse axes and θ is the rotation angle. In the first stage of the bubble detector, a regression CNN is trained that learns to extract the five fitting parameters from labelled images of bubbles.

3.1.1. Training data

Bubbles are relevant in a variety of technical applications. Consequently, experimental conditions can vary largely. However, these strongly affect the quality of the images and the distribution of pixel light intensity values. Thus, the training data set should cover as many different experimental parameters as possible so that both the shape regression CNN and the Faster RCNN can be used as generally as possible. The data set for the shape regression CNN consisted of about 100 000 images, 90 000 of which are randomly selected for training and the remaining 10 000 for validation. Images from three different sources were used for this purpose. Some exemplary images are provided in Fig. 2.

A quarter of the data was generated synthetically using the BubGAN bubble generator from Fu and Liu (Fu and Liu, 2019). The bubble swarm density, bubble size distribution and background intensity were randomly varied. BubGAN is based on measurements of a bubble swarm with a void fraction of about 0.1 and an average equivalent bubble size of about 3 mm in a relatively small water model. Thus, the backlight is bright and the distance between camera and bubbles is relatively small. Bubbles can be seen with a high resolution and the phase boundary and the bubble interior are distinct.

The second quarter were extracted from highspeed videos of bubble swarms in a 1:5 water model of a steel casting ladle. The model was filled to a height of 0.646 m and its diameter was 0.64 m. Bubbles were generated by a porous plug with a diameter of 0.02 m, eccentrically placed at r = 0.21 m. To further increase the

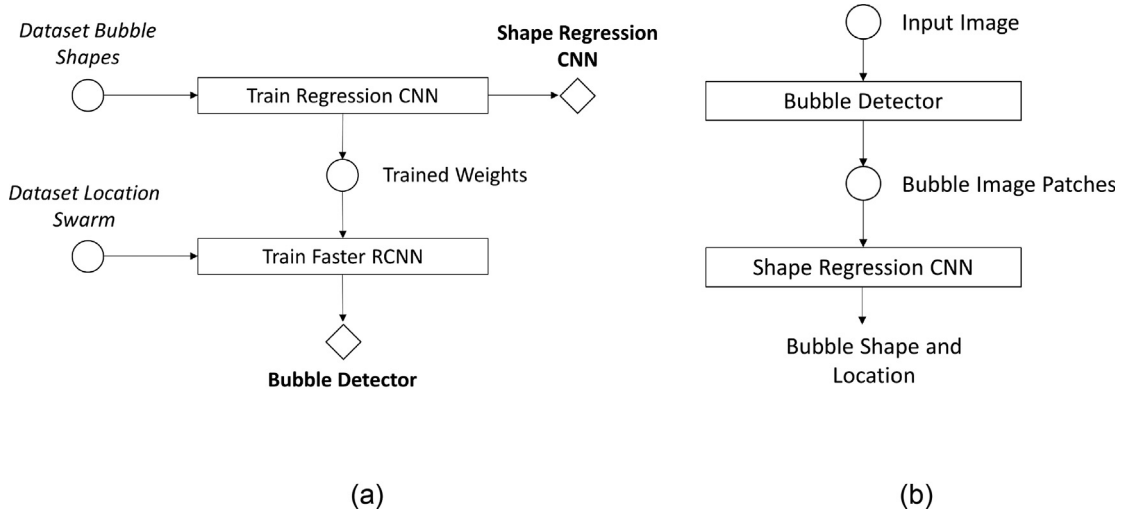


Fig. 1. Training (a) and image processing (b) stages of the proposed BubCNN workflow.

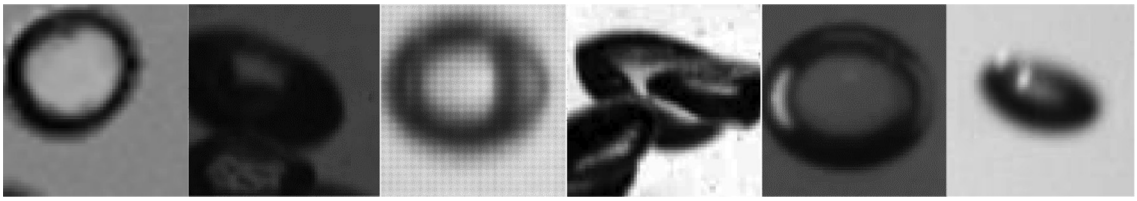


Fig. 2. Training images for shape regression CNN.

diversity of the data set, the camera aperture, the camera lens and the distance of the camera to the bubble column were varied. The average equivalent bubble size is between 3 mm and 5 mm with a void fraction up to 0.07 depending on the trial. The distances between the backlight, the bubble column and the camera are much larger than in BubGAN. Thus, the intensity gradients are less pronounced and the inside of the bubble is less distinct. In addition, the bubble column fluctuates radially. As a result, some of the recorded bubble outlines become blurred.

Another quarter of the data set was taken from experiments on the rise of single bubbles in different water-glycerin mixtures and bubble sizes between 2 mm and 8 mm. Here, images taken with two different highspeed cameras (Photron FASTCAM SA3 and Dalsa Genie Nano M1280) were used.

The last quarter was created by artificially modifying randomly selected images from the existing data set. In order to further increase the diversity, various artificial image augmentation techniques were carried out. This includes image noise with random strength, blurring by a Gauss filter and random intensity adjustment.

The images of individual bubbles were extracted by first constructing a bounding box around the bubble. The position and size of the box was slightly varied randomly to accommodate inaccurately positioned Faster RCNN bounding boxes from the previous process stage.

The bounding boxes were then transformed into a rectangular shape by expanding the shorter side to the length of the longer side. These square boxes were extracted from the original image to obtain a small image section containing the bubble. The section was resized to 64x64 pixels. By that, it is ensured that the shape regression CNN can handle bubbles of different sizes.

For each image patch of the data set, the five ellipse parameters were derived as target values. In BubGAN, these parameters were directly accessible. For the videos of the bubble column in the ladle model they were extracted by a traditional imaging procedure. Bubble tracking was used in order to avoid erroneous data. In this additional control step, all objects that are visible for less than 10 consecutive time steps were discarded.

3.2. Faster RCNN module

RCNNs are a class of object detectors in which the computational costs are significantly reduced compared to sliding window methods by not processing all image patches. Instead, a number of region proposals are created, which are then warped to the correct size and labelled by classification CNNs and refined by regression CNNs.

Faster RCNNs further increase computational efficiency by making the whole workflow fully convolutional. Here, all convolutional, activation and pooling operations of a pretrained CNN are applied to an input image of arbitrary size to obtain a final convolutional feature map. In this work, the weights learned during training the regression CNN were used. Faster RCNN employs the final convolutional feature map for two modules, a region proposals network (RPN) and a Fast RCNN module that evaluate the proposals.

3.2.1. RPN

In the first module, a RPN is used to generate rectangular proposals where bubbles might be located. For that, in the original work by Ren et al. (Ren et al., 2015) a convolutional Layer with 256 3×3 filters is slid over the last convolutional feature map.

For each position, a "pyramid of anchors" is created. The anchor pyramid consists of k rectangular anchors centered at the center location of the receptive field. The anchors vary in aspect ratio and size. For each location, all anchors are fed into a box-classification layer that outputs objectness scores for each anchor. The objectness score is a probability whether the anchor contains any object or not. A further box-regression layer is used to refine the region proposals. The anchors solve the problem of different scales. Bubbles can be seen in the image in different sizes. The reason is either that the bubbles have different physical sizes or that the distance between the plume and the camera can vary. That is accounted for by the pyramid of anchors.

3.2.2. Fast RCNN module

The resulting region proposals are further processed by a Fast RCNN module. The region proposals of arbitrary size are warped by region of interest (ROI) pooling so that they match the size requirements of the fully connected layers of the module. These warped regions are further processed by a classifier, outputting probabilities for different classes including background for each region proposal. A regression CNN further refining the proposed regions to bounding boxes. Instead of processing each region proposal individually the last convolutional feature map is used to reduce processing resources by shared computations.

A problem with RCNNs is that one object might be contained in different anchors. Thus, different overlapping bounding boxes are created for the same bubble. Therefore, a non-maximum suppression is applied as a post processing step. Bounding boxes that have an intersection-over-union (IoU) of at least 50% are considered to refer to the same bubble. Only one bounding box, that with the highest objectness score is kept. By that double detections can be avoided. On the other hand, correctly detected overlapping bubbles might be discarded.

The data set to train the Faster RCNN module is similar to those used to train the shape regression CNN though the requirements for both differ. While regression CNN required fixed sized image patches with ellipse parameters, Faster RCNN can process arbitrary sized images. In addition, the targets are bounding box parameters, x, y, width and height. The number of targets per images can differ too. A data set of 3750 images were gathered where 1500 images were generated by BubGAN, 1500 images were from experimental images of a bubble swarm and 750 images were from experiments for single bubbles. The images from BubGAN varied in void fraction and bubble size distribution. In addition, the images were randomly resized and some augmented with noise, blurring or contrast adjustment. The bubble swarm images were taken from experiments with different background lightning, camera lenses,

camera distances and swarm void fractions. The images from single bubble tracking were randomly cropped and resized. In addition, some were augmented with noise, blurring or contrast adjustment. The data set was randomly split so that 75% were used to train the network and 25% were used to evaluate its performance.

During training, positive and negative training examples are generated. Positive examples are generated from the anchors with the highest IoU overlap with a bounding box of the training data or for anchors which intersection-over-union with any bounding box exceeds a threshold value of IoU_{max}. Negative training examples are generated from anchors which intersection-over-union overlap is less than IoU_{min}. Since positive and negative training examples should be balanced to avoid a bias of the classifier, both threshold values are investigated in this work.

4. Analysis of hyperparameters and network architecture

The first module of the bubble detector is a shape regression CNN. Though it is used in the second stage in the final application, the Faster RCNN module is based on the pretrained weights of the regression CNN. Thus, optimal settings are discussed first.

4.1. Shape regression CNN

The accuracy of the shape regression CNN depends on various hyperparameters such as the network architecture or the optimization algorithm. These were systematically investigated in this paper. As a measure of accuracy, the relative deviation between the six outputs of the fully trained regression network and the validation targets was used. The most extreme values were ignored to reduce the impact of outliers with a total of 5 percent of all values being discarded. The mean relative deviations of the parameters were averaged to obtain a single value for the performance of the regression CNN. Since learning is a stochastic approach, the results differed within a certain range. For this reason, training was repeated five times for each set of hyperparameters. This prevents wrong conclusions being drawn from the scattering of the results. The mean relative error is given by:

$$\text{Mean relative error} = \sum_{k=1}^{6} \frac{\sum_{i=1}^{n,val} \frac{\left(y_i - y_i^p\right)^2}{y_i}}{n_{val}}$$

where y^i are the target values, y_p^i are the predicted values and n_{val} is the number of validation data points.

All networks were trained for 100 training epochs and an initial learning rate of 10^{-2}. The learning rate dropped every fifth epoch to 70% of its previous value. The weights were initialized by small random numbers.

4.1.1. Feature scaling and optimization algorithm

The target values of regression may differ by orders of magnitude. Therefore, they have to be rescaled in order to prevent the influence of individual parameters from being over- or under-represented during training. Either feature standardization or feature normalization can be used:

$$\text{Normalization}: \quad y_N = \frac{y - y_{min}}{y_{max} - y_{min}}$$

$$\text{Standardization}: \quad y_s = \frac{y - \mu_y}{\sigma_y}$$

where y_{min} and y_{max} are the smallest respectively the highest value of the data set, μ_y indicates its mean and σ_y its standard deviation.

Particular attention must be paid to the regression parameter θ. The rotation angle has a periodicity that can cause high errors during training although the prediction of the network was relatively precise. Owing to the assumed symmetry of the ellipse, the periodicity frequency is π. This periodicity can be considered by encoding the parameter θ:

$$t_{s1} = \sin 2\theta$$

$$t_{s2} = \cos 2\theta$$

After training, the values can be decoded to gain the rotation angle θ by:

$$\tan 2\theta = \frac{t_{s1}}{t_{s2}}$$

In this work, feature normalization, standardization and no preprocessing apart from angle encoding are compared. In order to ensure a comparability of the results, the predictions were retransformed after training and the relative error for all predictions were calculated. The averaged relative error is shown in Fig. 3(a). The results of the different training runs are indicated by the diamond shaped markers. The range of results is represented by the boxplots. The relative error is smallest for standardized targets. All training runs yield very similar results. The averaged error is about 0.075. Normalization yields slightly higher errors and a slightly higher variance. If the targets are not rescaled, the relative error doubles to 0.151. In addition, the results vary strongly.

Amongst data pre-processing, the optimization algorithm is an important factor for the accuracy of the regression CNN. Here, the most common learning algorithms Adam, RMSProp and stochastic gradient descent with momentum (SGDM) are compared. For that and all following studies, the targets were standardized. The same network architecture as for the preprocessing study was chosen. The results are shown in Fig. 3(b). It can be seen that the Adam optimizer yields very poor results, while RMSProp and SGDM are much better. SGDM yield the best results and was henceforth used.

4.1.2. Loss function and L2 regularization

As described above, the backpropagation optimization employs a loss function as a performance measure. Different loss functions can be used. Here, the mean squared error (MSE) and the mean absolute error (MAE) are compared.

$$\text{MSE} = \frac{\sum_{i=1}^{n} \left(y_i - y_i^p\right)^2}{n}$$

$$\text{MAE} = \frac{\sum_{i=1}^{n} \left|y_i - y_i^p\right|}{n}$$

where y^i are the target values and y_p^i are the predicted values.

Generally, using the mean absolute error is more robust to outliers but may miss the minimum since the gradients do not decrease around it. Decreasing learning rates can be used to solve this problem.

As shown in Fig. 4(a), the MAE loss function yields slightly better results than MSE loss. The difference is low though. This indicates that the data set contains very few outliers. Regularization is a technique to reduce overfitting of the training data by adding an extra term to the loss function which penalize large weight values. Here, different L2 regularization factors were tested. The results are shown in Fig. 4(b) on a logarithmic scale. It can be seen that the network is unable to learn the features if the regularization factor is too high. The network is underfitting the data and the validation and the training error are both high. In case of smaller regularization factors both errors drop to very low values. As expected, the training error is slightly lower than the validation error but the deviation is small. Thus, a regularization factor of 10^{-4} or lower is recommended.

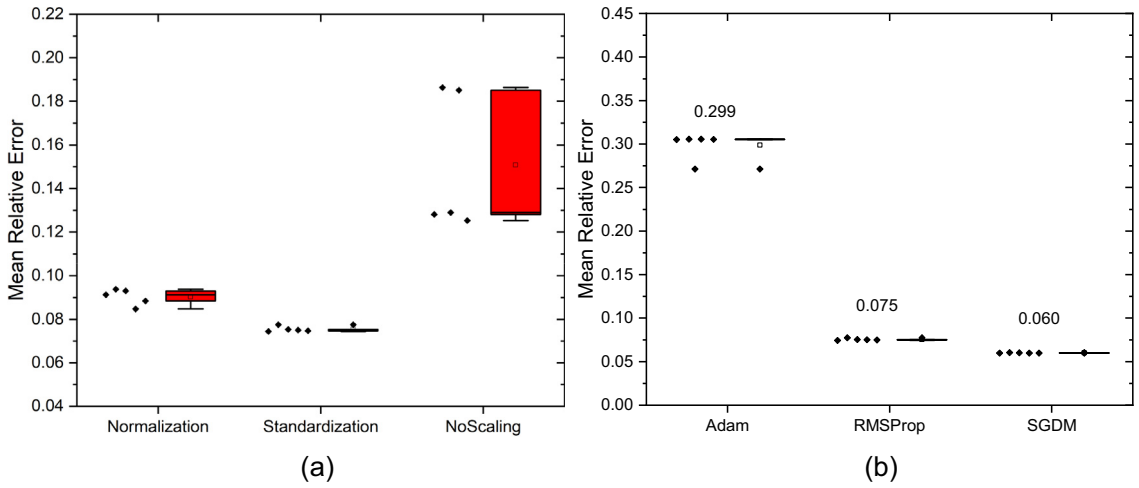


Fig. 3. Mean relative error in dependency of feature pre-processing (a) and optimizer (b).

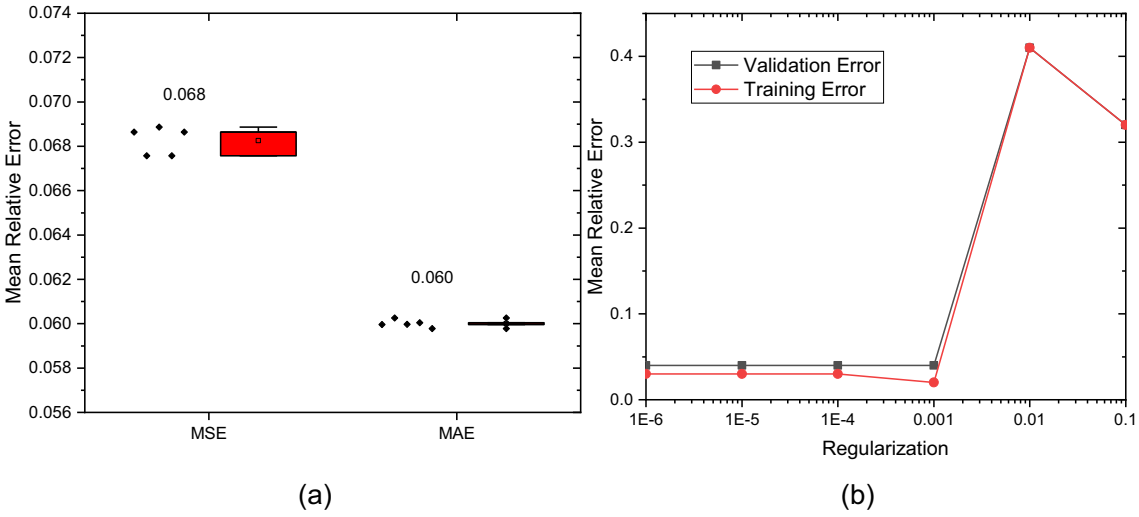


Fig. 4. Mean relative error in dependency of loss function (a) and regularization (b).

4.1.3. Architecture

The general structure of the shape regression CNN is given in Fig. 5. A convolutional section consists of one or more convolutional layers. Each convolutional and fully connected layer is followed by an activation layer, that is not shown for the sake of simplicity. Since the final fully connected layers were replaced when training a Faster RCNN, the number of neurons in this layer is set to 50 though the overall accuracy could be increased by increasing the number of neurons. By that relatively small value, the feature extraction by the convolutional layers need to be more precise which is be beneficial for the Faster RCNN.

The number of filters of a convolutional layer in a convolutional section will be examined. It was found, that steadily increasing the number of filters with the network depth is beneficial. Thus, independent of the initial number, the number of filters was doubled after each section. Initial numbers of filters of 4, 8, 16, 32, 64 and 128 were tested. In addition, the filter size was varied. The filter is given by n × n matrix where n was 3, 5, 7, 9, 12 or 15. Finally, the network depth was increased. For that, the number of convolutional layers in a convolutional section was increased. Section depth between one and five were tested. To reduce the number of parameters, the later was only conducted for two different filter sizes. Note that there is no such thing as an optimal architecture, because the results always depend on the dataset and hyperparameters. Still, some general insights can be gained by a systematic analysis.

Image Input [Image Size: 64 64 1]
Convolutional Section [Num. Filters, Filter Size, Repeats]
Max Pooling Layer [Pool Size: 2]
Convolutional Section [2 · Num. Filters, Filter Size, Repeats]
Max Pooling Layer [Pool Size: 2]
Convolutional Section [4 · Num. Filters, Filter Size, Repeats]
Max Pooling Layer [2]
Fully Connected Layer [Neurons: 50]
Fully Connected Layer [Neurons: 6]
Regression Layer

Fig. 5. Schematic network architecture of shape regression CNN.

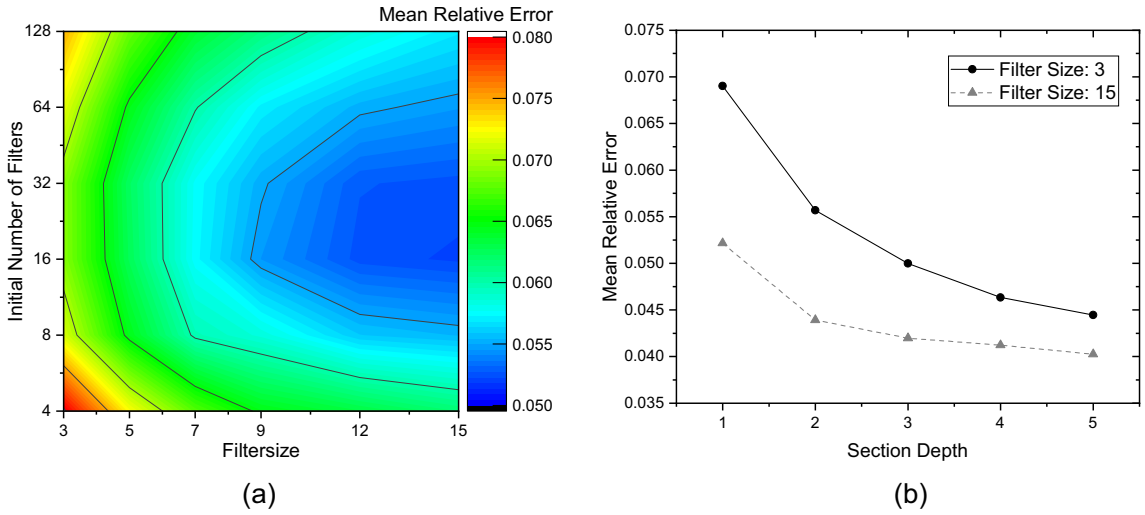


Fig. 6. Mean relative error in dependency of initial number of filters, filter size (a) and section depth (b).

The results are summarized in Fig. 6(a) and (b). It is apparent that increasing the filter size decreases the relative error. However, the effect declines with the filter size. Increasing the initial number of filters can have a negative effect. However, it is assumed that this is mainly because convergence takes longer and the number of training epochs was so small. The optimum initial number of filters is between 16 and 32. A more significant effect can be obtained by increasing the depth of a convolutional section. With that the effective filter size increases. That is a far more effective approach because there are far less weights to adjust during training.

The studies can be summarized by some advices for training the regression CNN. Before training the targets should be scaled. Standardization and normalization are both good choices, though standardization yields slightly better results. SGDM is the best tested optimization algorithm, Adam should not be used. MAE loss is slightly better than MSE, however it is important to use decreasing learning rates while using MAE. Regularization should be limited to very small values. Increasing the depth of the convolutional sections significantly decreases the relative error. A similar but smaller effect can be gained by increasing the filter size. Increasing the number of initial filters is not helpful though.

Some results of the shape regression CNN are shown in Fig. 7. It can be seen that the fit is almost perfect in case of the bubbles being isolated as individual bubbles. In case of bubbles overlapping the deviation increases with the degree of overlap.

4.2. Faster RCNN

Like for the regression CNN, the optimal hyperparameters were investigated for the Faster RCNN module. Different optimizers were tested as well as the number of training epochs and the number of region proposals. Finally, the most appropriate overlap ratios to create positive and negative training samples were analyzed.

The trained detectors were evaluated using the mean average precision (mAP) and the F1 score as a performance metric:

$$F1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

where the recall a measure of how much of the real positives were found:

$$\text{Recall} = \frac{\text{Truepositive}}{\text{True positive} + \text{False negative}}$$

And the precision measures the accuracy of predictions:

$$\text{Precision} = \frac{\text{TruePositive}}{\text{True positive} + \text{False positive}}$$

For both metrics, a true positive is a predicted bounding box with an IoU with a bounding box of the labeled data set of at least 50%.

4.2.1. Optimization algorithm

Like for the regression CNN, three different optimizers were tested, RMSprop, SGDM and Adam. All detectors were trained with the 4-step alternating training method (Ren et al., 2015). Each step was conducted for 50 training epochs and an initial learning rate of 10^{-4}. The learning rate was halved every 10 epochs. 1000 region proposals were evaluated and the overlap thresholds were set to $IoU_{max} = 0.7$ and $IoU_{min} = 0.3$.

The mean average precision (mAP) and the F1 score, evaluated on the validation data set, are given in Table 1. Note that a higher mAP value and F1-score correspond to more accurate results. It can be seen that the best detection results can be achieved with the

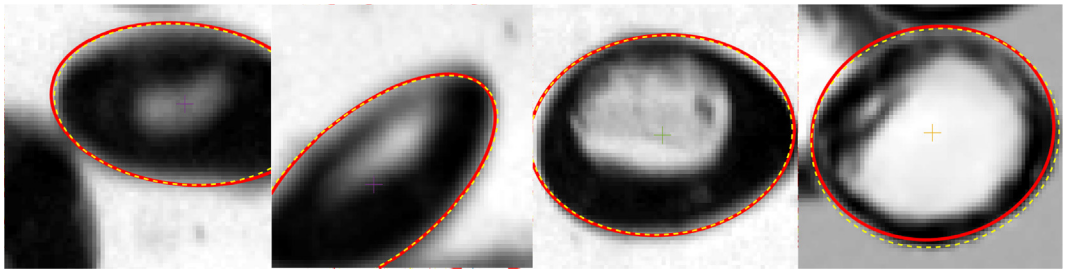


Fig. 7. Bubbles with training ellipses (yellow dashed) and predicted ellipses (red). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Table 1
Detector performance in dependency of the optimizer.

	mAP(@0.5)	F1	Recall	Precision
SGDM	0.59	0.66	0.62	0.70
RMSProp	0.73	0.78	0.80	0.77
Adam	0.79	0.80	0.81	0.80

Table 2
Detector performance in dependency of the number of region proposals.

	mAP(@0.5)	F1	Recall	Precision
1000	0.79	0.80	0.81	0.8
2000	0.84	0.89	0.85	0.95
infinite	0.84	0.90	0.85	0.97

Adam optimizer. In contrast to the regression CNN module, the SGDM optimizer is not advisable.

4.2.2. Training epochs and number of region proposals

To test if the results can be improved by increasing the duration of training, the number of training epochs was increased to 150, using the Adam solver. A mAP value of 0.78 and a F1-score of 0.81 were achieved. That is very similar to the results obtained with 50 training epochs. Thus, it can be concluded that increasing the number of training epochs is unnecessary.

In terms of the number of region proposals generated by the RPN, values of 1000, 2000 and an unrestricted number of regions were analyzed. The results are summarized in Table 2. Given the high number of bubbles on an image of a bubble swarm, sampling of 1000 regions is insufficient. Instead, an infinite number of regions to sample is recommended.

4.2.3. Overlap thresholds for training data generation

Finally, the effect of the overlap thresholds for training sample generation was investigated. Negative training examples are generated from anchors which intersection-over-union overlap is less than IoU_{min}. Here, values of 0.05, 0.1, 0.2 and 0.4 were tested. Positive samples are generated from anchors which intersection-over-union with any bounding box exceed a threshold value of IoU_{max}. Values of 0.4, 0.5, 0.6, 0.7, 0.8 and 0.9 were tested. The results are given in Table 3. The first value indicates the mAP(@0.5), the following the precision, and the recall respectively. It was found that increasing the negative overlap increases the precision for the whole investigated range. That increase of the true positive ratio is because increasing the negative threshold will cause more negative samples. Consequently, the detectors classifier is biased towards negative results and will only output a positive in case of a very high activation. Increasing the positive threshold has a similar effect, because the skewness of the data will be further increased. For the given dataset, detection performance peaks for a negative overlap of 0.2 and a positive overlap of 0.4 or 0.5.

It can be assumed, that the positive and negative training samples are almost balanced. In case the positive overlap is further increased, the recall decreases and the classifier has a higher missing rate. Note, that the optimum depends upon the postprocessing overlap value of 0.5 and the properties of the data set. The void fraction might be particularly important. Training images with higher void fractions might work better with lower negative threshold values. However, it can be assumed that the optimum for bubble flows will always be in the found range.

One interesting result is that the performance of the Faster RCNN module is mainly influenced by the overlap threshold values set for the creation of training data. Taking the trends indicated in Table 3, the detector can be customized to the needs of the researcher. In case bubble detection is used for a bubble size distribution, the precision is more important as the recall because in most cases statistically significant distributions can be obtained without analyzing all bubbles. In that case, increasing the value for IoU_{min} and IoU_{max} will be a good choice. For bubble swarm tracking on the other hand, a higher recall value is important. The reduction of precision is to be overcome by the tracking assignment, that can filter out most false positives.

5. Final assembly

The two trained modules were assembled in a program that perform all steps of the full workflow. This program is made publicly available on GitHub: https://github.com/Tim-Haas/BubCNN. The results are shown in Fig. 8. First, an image is processed by the Faster RCNN module that detects bubbles and outputs bounding boxes (yellow). Then the resulting bounding boxes are resized to 64x64 pixel patches and processed by the shape-regression CNN. Finally, the results that are still standardized are decoded by the mean and standard deviation. Processing a 1024 × 1024 grayscale image with approximately 200 bubbles takes about 35 s on a CPU (AMD Ryzen 7 1700 X) and 3.5 s on a GPU (Tesla V100 SXM2). In Fig. 9 different experimental settings and detected bubbles are shown. In contrast to traditional methods, the machine learning approach can deal with those different settings without manual modifications. The accuracy is high, though it is important to notice that it is always lower than those of customized traditional image processing. The reason for that is that the training data contain the inaccuracies of the image processing or manual marking of the bubbles. In addition, the training error is added to the inaccuracies. The deviation between these errors can be significantly reduced through by large data sets and appropriate learning parameters. The most noticeable error is that some ellipses are 90° out of phase. That is caused by the angle encoding for which rotation angles of exactly 45° and 135° are indistinguishable. However, if BubCNN is used to determine the bubble size distribution, the rotation angle is not of interest. In case of it being used for bubble tracking, that error can be corrected because each bubble is tracked through multiple frames and it is highly unlikely that the rotation angle is 45° or 135° on each frame. Another mis-

Table 3
Detector performance in dependency of the positive and negative overlap for training data generation.

IoU_{max} \ IoU_{min}	0.05	0.1	0.2	0.4
0.4	0.87/0.86/0.93	0.93/0.91/0.95	0.95/0.95/0.96	0.95/0.98/0.94
0.5	0.90/0.85/0.94	0.93/0.92/0.95	0.95/0.98/0.95	0.93/0.98/0.94
0.6	0.87/0.83/0.91	0.89/0.86/0.91	0.93/0.97/0.94	0.89/0.98/0.90
0.7	0.80/0.76/0.87	0.82/0.81/0.86	0.87/0.97/0.89	0.85/0.99/0.86
0.8	0.73/0.72/0.78	0.69/0.86/0.81	0.73/0.95/0.79	0.59/0.99/0.68
0.9	0.36/0.68/0.48	0.33/0.66/0.44	n/a	n/a

*mAP/Precision/Recall.

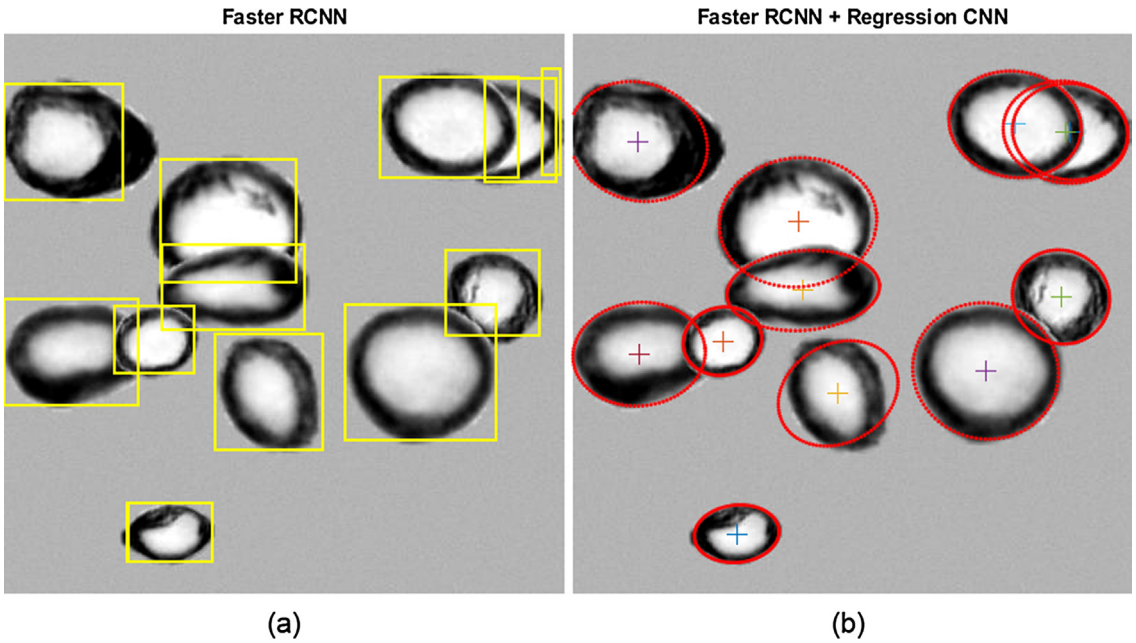


Fig. 8. Example of detection process. Detection (a) and regression (b) stage.

take is that some bubbles are counted twice. That can be explained by an insufficient non-maximum suppression.

In Fig. 10, the mAP, precision and recall are shown for different three-dimensional void fractions. The test images were created artificially by BubGAN. For each void fraction 300 images were evaluated. In accordance with the literature (Fu and Liu, 2016; Bröder and Sommerfeld, 2007; Lau et al., 2013), the void fraction is defined by the volume of all bubbles divided by the total volume. Note that the void fraction also depends on the third dimension that is along the camera axis. The detection accuracy depends mainly on the fraction of area that is covered by bubbles in the two-dimensional projection. That means that the accuracy for swarms with a large expansion in the third dimension is lower than for the same three-dimensional void fraction with a smaller expansion. Owing to that, the large deviations in the accuracy of different traditional imaging processes reported in the literature can be explained. Here the third dimension has an expansion of 1 mm. For the training images, the void fraction was increased from 0.025 to 1.5 by steps of 0.05. For very low void fractions, all bubbles are visible as single bubbles. The detector detects all bubbles and without false positives. With increasing void fraction, the bubbles are visible as bubble clusters. The mAP decreases linearly, mainly because the missing rate increases. The precision is less affected by the void fraction. For very high void fractions, almost all bubbles are part of a cluster. Still the mAP is high and the missing rate is only of about 25%. It is important to note that the missing rate could be significantly improved by a more appropriate non-maximum suppression. Especially for high void fractions, bubbles overlap by more than 50% and are thus discarded during post-processing. In future versions of BubCNN other non-maximum suppression techniques should be developed and employed.

6. Transfer learning module

Feedback from researchers that tested the pretrained BubCNN confirmed that it yields accurate results if the tested bubble images

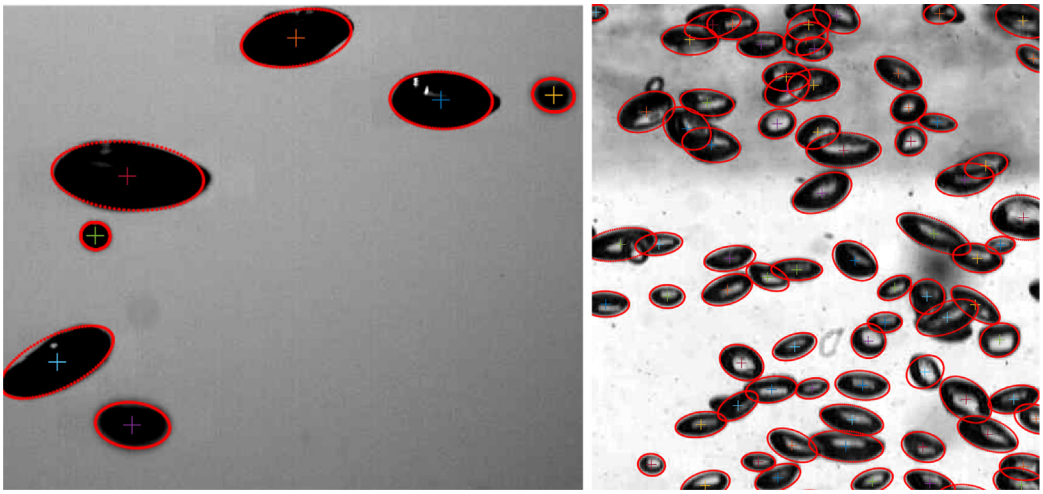


Fig. 9. Example results for different experimental setups.

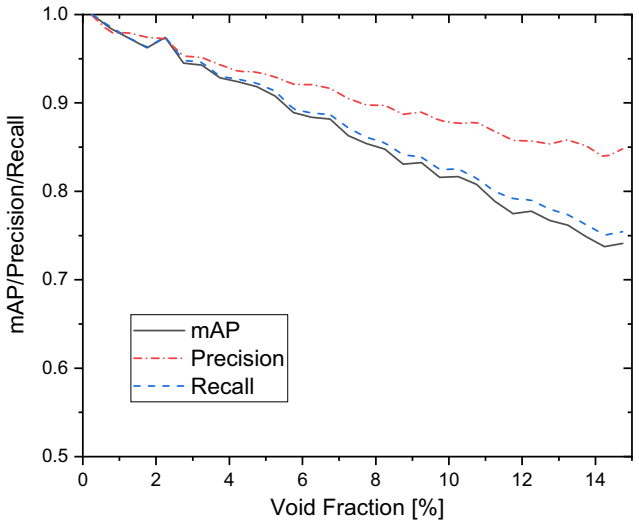


Fig. 10. mAP, precision and recall of BubCNN as a function of the void fraction.

resemble those used for the training data set. This can be understood as a proof of concept. Nonetheless it was also reported that the accuracy is significantly reduced if the experimental images differ strongly from the training data. That indicates that though augmentation was used to account for altered experimental conditions, the training data set is biased towards images that have similar experimental conditions as the training images. The whole diversity that images of bubbly flows can have is not captured in the data set yet. It can be concluded, that a universally applicable detector would require a very large and diverse data set that cannot be created by a single lab because of limitations like experimental setups or cameras. Because of that, a transfer learning module was developed that allow researchers to customize BubCNN to their own images. Like the BubCNN detector, the transfer learning module was made publicly available on GitHub: https://github.com/Tim-Haas/BubCNN. Through that BubCNN can be used for all kind of images and a database can be built up if the transfer learning data is sent to the authors. For the transfer learning module, the MATLAB App Designer was used. The graphical user interface is shown in Fig. 11.

The user can upload an image or videos of the bubbles BubCNN should be customized for. The image can be cropped. The idea is that bubbles have to be marked with ellipses in a semi-automatic procedure. To reduce the workload, bubbles can be automatically identified by two different methods. The first method employs the current version pretrained version of BubCNN. The second method uses a simple, parameter-free image processing procedure that consists of gradient analysis, k-means segmentation and ellipse fitting. The proposed ellipses can be manually modified or deleted and completed by adding new ellipses. This is done by dragging and dropping ellipse objects as shown in Fig. 12. The transfer learning module automatically creates data sets to train the Faster RCNN and the shape regression CNN module by using the ellipse data. Augmentation is used to increase the amount of training data.

In the actual transfer learning step, the pretrained modules are retrained using the new data set and a lower initial learning rate. It was found that marking the bubbles on a single frame was sufficient to successfully retrain the BubCNN detector. For about 150 marked bubbles, the retraining process took about one hour on a GPU (Tesla V100 SXM2).

Even though some work is required to create the data set in the semi-automatic workflow, no expert knowledge about image processing is required any more. Therefore, measurements in bubbly flows can be used by a broader range of researchers. Users are requested to share the transfer learning datasets with the authors to contribute to a bubble database for more comprehensive future versions of BubCNN.

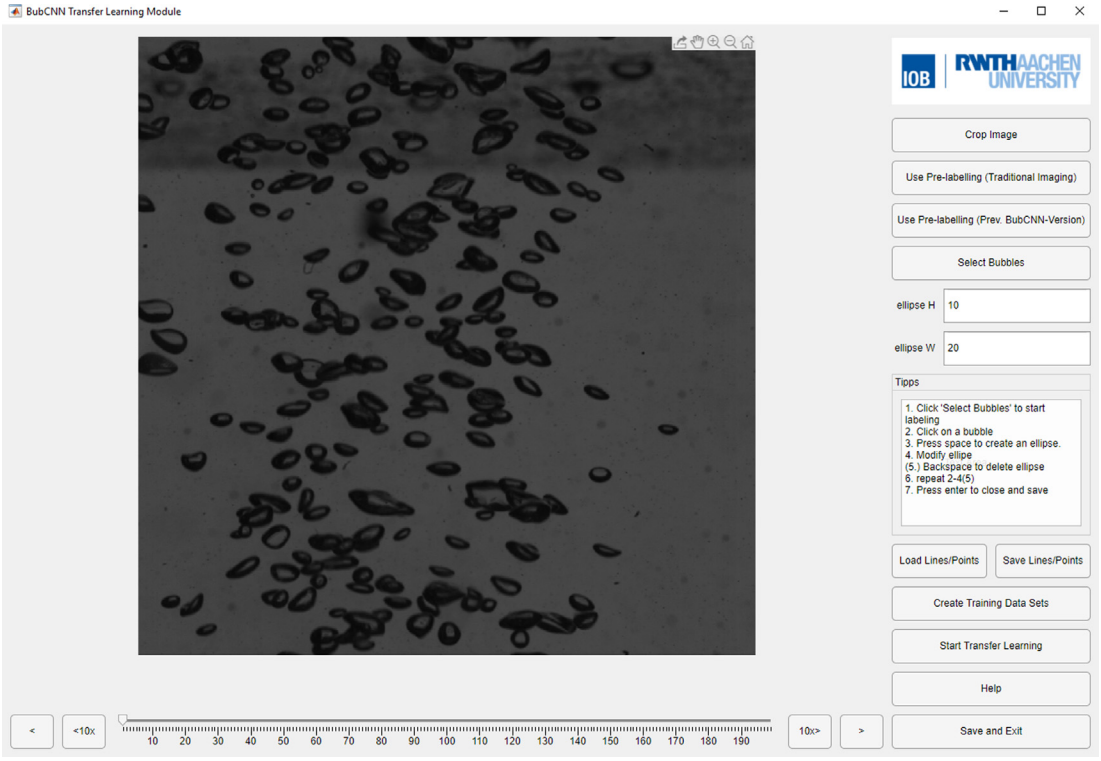


Fig. 11. Graphical user interface of transfer learning module.

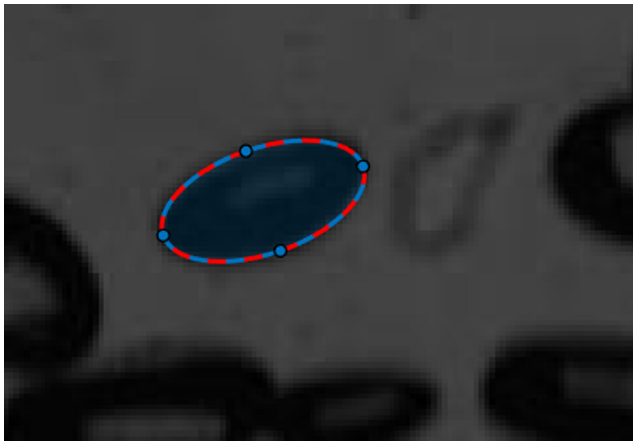


Fig. 12. Manual drag and drop of an ellipse in the transfer learning module.

7. Discussion

State of the art detectors for bubbles on images, based on traditional image processing, were found to be difficult to generalize. They yield highly accurate results for given experimental conditions, yet cannot be used in different set-ups. In this work, a workflow for a machine learning based bubble detector is proposed to overcome this problem. The procedure is two-staged, including a Faster RCNN module to locate bubbles and a shape regression CNN for bubble shape parameters. The final procedure was named BubCNN and made publicly available on GitHub.

Accurate results could be achieved for all images that resemble the training data to a certain degree. This can be understood as a proof of concept. The training data included different experimental setups. Therefore, the BubCNN workflow is already more general than previous bubble detectors, based on traditional image processing methods. However, it was found that it is still not general enough to account for all different experimental conditions. Even though data augmentation techniques were used, the variety of high-speed cameras, experimental setups, light sources and camera lenses available in one the lab is small compared to the number of settings possible worldwide. Therefore, the current data set is biased towards the settings used for training. To account for that, an additional semi-automatic transfer learning module was developed allowing to customize the BubCNN workflow for different bubble images. In contrast to current workflows, the transfer learning module requires no expert knowledge about image processing and can be used by a wide range of researchers. Users are requested to send their training data to the authors to help building up a bubble data base. With that, more general versions of BubCNN can be trained and shared in the future.

An analysis of optimal hyperparameters revealed some useful guidelines about which training conditions to choose. For example, it was found that the SGDM optimizer is most appropriate to train the shape regression CNN while Adam optimization is the best choice for training the Faster RCNN module. The negative and positive overlapping thresholds for the generation of training samples were found to have an important impact on the performance of the Faster RCNN. It is assumed that the optimal values depend upon different parameters such as the void fraction. In future works, an empiric correlation should be found so that the transfer learning module automatically determine optimal thresholds.

An intrinsic problem of the detector is the assumption of ellipsoidal-shaped bubbles. This assumption is inaccurate in case of higher Eotvos and Reynolds numbers. While conventional image processing tools can theoretically reconstruct every shape, the machine-learning approach is less flexible. To expand the range of validity, the shape regression CNN could be extended so that the shorter semi-axis b is divided into two independent semi-axes b^- and b^+. That accounts for slightly deformed ellipsoids. Another machine learning based approach that can process arbitrary shapes is semantic segmentation. In this relatively novel branch of CNNs, every pixel is classified individually. The techniques might be a promising approach for future developments. However appropriate interpolation methods need to be applied overlapping bubbles.

Analyzing the detectors performance in dependency of the void fractions showed that the current non-maximum suppression is unsuitable in case of highly overlapping bubbles. Correctly detected bubbles are frequently discarded. The performance of BubCNN can be significantly increased by a customize non-maximum suppression post-processing. Therefore, a more appropriate method should be a focus of future developments.

So far, the implementation has been made in MATLAB. Thus, the usage of BubCNN is restricted to users that have access to that commercial programming framework. To overcome that, a reimplementation in an open-source programming framework like Python could be useful.

8. Conclusion

A novel, machine learning based bubble detection workflow names BubCNN was proposed. The workflow includes a Faster RCNN module to detect bubbles and a shape regression CNN module to approximate the bubble shapes by ellipses. Through this approach, the efficiency of modern RCNNs and the accuracy of regression CNNs are combined. The accuracy of the Faster RCNN module was increased by employing pretrained layers of the regression CNN. BubCNN shows better abilities to generalize compared to state-of-the-art image processing procedures. Owing to that, it is a tool that can be applied to a wide range of experimental setups and conditions and saves researchers from having to program a new evaluation algorithm repeatedly. It was made publicly available by sharing the program on GitHub. Researchers are invited to test the detector and help increasing its applicability by sharing their feedback and experimental data.

CRediT authorship contribution statement

Tim Haas: Conceptualization, Methodology, Software, Validation, Investigation, Data curation, Visualization. **Christian Schubert:** Software, Data curation. **Moritz Eickhoff:** Supervision, Funding acquisition. **Herbert Pfeifer:** Project administration, Funding acquisition.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

The authors gratefully acknowledge the support from the German Research Foundation (Deutsche Forschungsgemeinschaft (DFG)) within the project RU 2050/2-1. Simulations were performed with computing resources granted by RWTH Aachen University under project nova0032. In addition, the authors are grateful to Thomas Hay and Ville-Valtteri Visuri for their valuable comments on this manuscript.

References

Haas, T., Eickhoff, M., Pfeifer, H., 2019. In: International Conference on Modeling and Simulation of Metallurgical Processes in Steelmaking. Toronto. 2019.

Fu, Y., Liu, Y., 2016. Int. J. Multiphase Flow, 84.

Bröder, D., Sommerfeld, M., 2007. Measurement Sci. Technol. 18, 8.

Ferreira, A., Pereira, G., Teixeira, J., Rocha, F., 2012. Chem. Eng. J., 180

Karn, A., Ellis, C., Arndt, R., Hong, J., 2015. Chem. Eng. Sci. 122, 240–249.

Lau, Y., Sujatha, K.T., Gaeini, M., Deen, N., Kuipers, J., 2013. Chem. Eng. Sci. 98, 203–211.

Zafari, S., Eerola, T., Sampo, J., Kälviäinen, H., Haario, H., 2015. IEEE Trans. Image Process. 24, 12.

Hukkanen, J., Hategan, A., Sabo, E., Tabus, I., 2010. Segmentation of cell nuclei from histological images by ellipse fitting. 2010 18th European Signal Processing Conference. IEEE.

Talbot, H., Appleton, B., 2002. In: Proceedings of ISMM2002.

Zafari, S., Eerola, T., Sampo, J., Kälviäinen, H., Haario, H., 2016. Segmentation of partially overlapping convex objects using branch and bound algorithm. Asian Conference on Computer Vision. Springer.

Kaur, S., Mittal, P., 2017. Int. J. Adv. Res. Comput. Sci. 8, 5.

Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., 2015. Int. J. Comput. Vis. 115, 3.

Poletaev, I., Pervunin, K., Tokarev, M., 2016. Artificial neural network for bubbles pattern recognition on the images. Journal of Physics: Conference Series. IOP Publishing.

Ilonen, J., Juránek, R., Eerola, T., Lensu, L., Dubská, M., Zemčík, P., Kälviäinen, H., 2018. Pattern Recognition Lett. 204.

Fu, Y., Liu, Y., 2019. Chem. Eng. Sci. 204, 35–47.

Clift, R., Grace, J.R., Weber, M.E., 2005. Bubbles, drops, and particles. Courier Corporation.

Ren, S., He, K., Girshick, R., Sun, J., 2015. Faster r-cnn: Towards real-time object detection with region proposal networks. Adv. Neural Inform. Process. Syst.